



Security Assessment



Aave Stable Vaults

January 2026

Prepared for Aave Labs

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	4
Assessment Methodology.....	6
Threat and Security Overview.....	7
Findings Summary.....	11
Severity Matrix.....	12
Detailed Findings.....	12
High Severity Issues.....	14
H-01: Slippage in deposit to strategy allows users to drain the vault.....	14
Medium Severity Issues.....	17
M-01: Strategy withdrawal mechanism is flawed due to not considering maxRedeem().....	17
M-02: Failed fund processing in CcipAdapter allows arbitrary callers to manipulate deposit amounts via replayFundsReceiving.....	22
M-03: Withdrawals can be DoS'ed by frontrunning the nonce usage.....	25
M-04: Missing deposit price check allows extracting value from the vault by depositing a depegged asset.....	27
Low Severity Issues.....	30
L-01: Missing deposit limiter mechanism.....	30
L-02: Malicious strategies cannot be distrusted and they can inflate TVL.....	31
L-03: _areRemainingSharesRedeemable() calculates best case scenario instead of worst.....	33
L-04: An attacker can force requestWithdrawal(user, 0) to revert if the caller is the only user in the protocol.....	35
L-05: TVL can temporarily be incorrect.....	37
L-06: Manager cannot rebalance and limit losses in case of a stablecoin depeg.....	39
L-07: Users can forcefully rebalance funds to default strategy.....	41
Informational Severity Issues.....	45
I-01: _setUserRate() could return early instead of reverting.....	45
I-02: Removing a strategy can be DOSed if not done in a multicall.....	46
I-03: Some functions do not need to be payable.....	47
I-04: Use constant instead of hardcoded value.....	49
I-05: Loops in FundsHandler can be optimized.....	50
I-06: Interest accrual can be optimized.....	52
Disclaimer.....	53
About Certora.....	53

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Aave Stable Vaults	https://github.com/aave/stable-vault https://github.com/aave/Aave-Vault	c5f16f1ffe492d PR#112	Solidity

Project Overview

This document describes the security review of **Aave Stable Vaults**. The work was undertaken from **January 5th, 2026** to **January 30th, 2026**.

The following contract list is included in our scope:

```
Aave-Vault/src/ATokenVaultMerkleRewardClaimer.sol
Aave-Vault/src/ATokenVaultStorage.sol
Aave-Vault/src/interfaces/IATokenVaultMerkleRewardClaimer.sol
Aave-Vault/src/dependencies/merkl/DistributorInterface.sol
stable-vault/src/bridging/across/IAcrossSpokePoolV3.sol
stable-vault/src/bridging/across/IAcrossV3Receiver.sol
stable-vault/src/bridging/ccip/CcipAdapter.sol
stable-vault/src/bridging/BaseBridgeAdapter.sol
stable-vault/src/core/accounting/AccountingChainGateway.sol
stable-vault/src/core/accounting/StableVault.sol
stable-vault/src/core/accounting/FundsHandler.sol
stable-vault/src/core/earning/EarningChainGateway.sol
stable-vault/src/core/ious/IouToken.sol
stable-vault/src/core/ious/IouTokenManager.sol
stable-vault/src/core/Allocator.sol
stable-vault/src/core/BaseChainGateway.sol
```

```
stable-vault/src/libraries/AssetLib.sol
stable-vault/src/libraries/MathLib.sol
stable-vault/src/misc/Multicall.sol
stable-vault/src/misc/RescuableAssets.sol
stable-vault/src/misc/TransferHelperClient.sol
stable-vault/src/periphery/AssetRegistry.sol
stable-vault/src/periphery/Swapper.sol
stable-vault/src/periphery/TransferHelper.sol
stable-vault/src/periphery/WithdrawalPolicy.sol
stable-vault/src/types/Constants.sol
stable-vault/src/types/Errors.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Stable Vaults is a semi-fixed rate vault protocol developed by Aave Labs. It allows users to deposit assets into "SubVaults" that offer a specific per-second interest rate, enabling predictable yield generation. Deposited funds are allocated to yield-generating strategies (e.g. Aave). The system tracks each user's original principal independently from accrued interest. In the event the vault cannot meet its total interest obligations, users are prioritized to withdraw at least their original deposit amount.

The system allows users to deposit whitelisted USD denominated stable coins on the accounting (canonical) chain. The depositor is minted vault shares, which are not transferable; funds are automatically deployed to the default strategy, when set.

The withdrawal process has two steps:

1. Vault share holders first request withdrawal of principal and interest – they are only allowed to withdraw interest if the protocol is solvent. Withdrawal requests burn vault shares and mint IOU tokens. This stops interest accrual and locks the USD value of the withdrawal.
2. IOU token holders can then redeem it for whitelisted USD denominated stable coins, minus a withdrawal fee. This can be done on the canonical chain; or IOUs can be bridged to any of the earning chains and redeemed there. There is a default withdrawal fee charged from all withdrawals – this fee can be overridden if the user provides a signed message from the

whitelisted signer. The withdrawal attempts to use idle funds not deployed to any strategy; if insufficient, it attempts to withdraw from the default strategy; if insufficient, from other strategies.

The system considers one unit of whitelisted stable coins to be worth 1 USD, and does not track deposit per asset back to users.

Deposits accrue a fixed-rate yield, set by governance for each user.

The funds deposited by users are automatically deployed to the default strategy. Governance whitelisted addresses can manage the funds, swapping the tokens held; or rebalancing the strategies to which these tokens are deployed.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Scope and Trust Assumptions

The review focused on logic errors and vulnerabilities in the smart contract code, including accounting-chain and earning-chain interactions, vault share and IOU accounting consistency, interest accrual correctness, defensive rounding, ERC-4626 strategy integration, withdrawal solvency, and asset trust/distrust handling. The review was conducted through manual analysis, invariants, and hypotheses.

The following are treated as trust assumptions, that must hold for the protocol to operate safely:

- Blockchain liveness and integrity: blocks are produced continuously, valid transactions are accepted, invalid ones rejected, fees remain reasonable.
- Bridge and cross-chain messaging integrity: the underlying messaging layers deliver messages reliably and in order, and do not drop, forge, or maliciously reorder cross-chain calls. The audit verified the protocol's integration with these layers only, not their internal security.
- Oracle correctness: the price feeds used for peg and depeg detection are timely, accurate, and manipulation-resistant. The audit verified integration only, not feed quality.
- Strategy integrity: external yield strategies (e.g., Aave) conform to the ERC-4626 standard, remain solvent, and are not malicious or arbitrarily upgradeable. The audit verified the vault's interaction with strategies, not the strategies themselves.
- Admin and governance integrity: multisig keys are secure, the execution time-lock is honored, and governance proposals are not malicious.
- Manager honesty within bounds: the semi-trusted manager performs rebalancing, rate-setting, surplus claims, and cross-chain fund movement without abusing its privileges. It is assumed unable to claim principal or cause principal loss, but capable of affecting yield and operational outcomes.
- Signer integrity: the whitelisted fee-discount signer's key is secure and is used only to sign legitimate withdrawal-fee overrides.
- Off-chain monitoring and response: monitoring systems detect depegs, anomalous balances, and malicious strategies, and operators act promptly, since deposits and withdrawals are permissionless and can deplete value before manual intervention.

System Invariants

The system is designed to preserve the following invariants under all conditions:

- Users can always redeem IOUs for their Original Deposit.
- IOUs are minted for interest only up to trusted surplus above guaranteed obligations.
- Admin may claim surplus only, defined as total assets minus (total principal deposits + total interest accrued).
- TVL cannot be artificially inflated.

Roles

The following roles carry distinct trust levels that must hold for the protocol to operate safely:

- Admin: fully trusted entity operating through a multisig account with an execution time-lock on each operation.
- Manager: semi-trusted entity responsible for rebalancing, setting interest rates, claiming surplus, and moving funds across chains, but unable to claim principal or cause principal loss.

Security Considerations and Threat Analysis

Deposit Integrity. A deposit must never degrade system solvency or shift costs onto existing users. Threats include:

- Deposits that harm the system or other depositors under any scenario.
- Deposit of depegged assets in the absence of a price check, rendering the system insolvent.
- Deployment slippage to the default strategy borne by the vault rather than the depositor.

Denial of Service. Critical actions, especially withdrawals, must remain executable at all times. Threats include:

- Blocking, stalling, or forcing retries of withdrawals or other critical flows.
- Nonce or replay-based DoS of discounted-fee withdrawals (e.g., front-running to consume a single-use signed message).

Access Control. Only authorized roles may perform privileged actions. Threats include:

- Bypassing access restrictions on protected functions.
- Spoofing or replaying signed messages and permissions.

Cross-Chain Integrity. Cross-chain accounting and messaging must be tamper-resistant and reliable. Threats include:

- Manipulating total TVL via cross-chain snapshots.
- Spoofing an earning chain's reported TVL or maliciously minting IOUs through cross-chain components.
- Cross-chain transfers or messages that can be blocked, stuck, or maliciously retried.

TVL and Interest Manipulation. TVL and accrued interest must reflect the true economic state. Threats include:

- Artificial inflation or deflation of TVL, impacting principal and interest claims.
- Manipulating the interest-rate mechanism so the effective rate diverges from the intended rate.

User-to-User Harm. No user action should harm other users or elevate systemic risk. Threats include:

- Actions that damage other users' claims or increase the risk of TVL loss.

Asset and Strategy Risk Management. The protocol must be able to retire dangerous components. Threats include:

- Inability to remove a heavily depegging asset or a malicious strategy to limit risk and losses.
- Limited admin-controlled mechanisms to minimize exposure when an asset or strategy becomes dangerous.

Trust Model and Design Decisions

The behaviors below follow directly from the trust assumptions and design goals stated above. They are not implementation vulnerabilities; they define the boundary within which those

assumptions must hold. They are documented here so that integrators and users understand where value depends on each assumption and what mitigations bound the residual risk.

Distrust coupled with deposit disablement (*relates to: Oracle correctness; Off-chain monitoring and response*). When an asset loses its peg, the first line of defense is automatic: the deposit price check (`IPriceOracle.validatePrice`) rejects any deposit of an asset whose price has fallen below the minimum valid price, without requiring human action. As a secondary measure, the asset can be distrusted. The `AssetRegistry` links these actions, so distrusting an asset also disables its deposits and swap output, and the asset must be trusted again before those can be re-enabled ([PR#194](#)).

Withdrawals of a distrusted asset stay enabled by design, so that users can always exit their position; blocking them would trap user funds, which is the worse outcome. The only remaining gap is a sudden depeg that occurs between oracle price updates. During that short interval the on-chain price is briefly out of date, so a deposit could pass before the feed catches up. This window is limited by how often the oracle updates, and off-chain monitoring can trigger an immediate on-chain distrust to disable deposits within it.

Reentrancy safety of listed components (*relates to: Asset and Strategy integrity*). Withdrawal accounting decrements obligations before the external asset and strategy transfers complete, so a reentrant listed token or strategy could re-enter and bypass the solvency check. Listed tokens and strategies are assumed to be non-reentrant and vetted before listing, so this precondition should not arise. As defense-in-depth beyond that assumption, `nonReentrant` guards were added to `deposit()`, `requestWithdrawal()`, and `executeWithdrawal()`, and to the share-transfer helpers ([PR#136](#)).

Early-redeemer ordering during a deficit (*relates to: Off-chain monitoring and response*). The system deliberately does not socialize losses; during a deficit it serves earlier redeemers first (Design Decision #6). A user who redeems just before a distrust action or a drop in TVL is redeeming their own position earlier in the queue, the same bank-run dynamic present in any pooled vault, rather than extracting value that was never theirs.

The impact falls on users who exit later. Because the available assets are paid out in order, users who redeem after the deficit appears may recover only their principal, without accrued interest,

and in a severe deficit could recover less than users who exited earlier. The loss is therefore borne by the slowest redeemers rather than shared across all users.

Operationally, emergency actions are submitted through private channels (e.g., Flashbots) and chain-balance oracle latency is minimized to narrow the window in which this ordering can be exploited. An enforced time delay between the request and execution of a withdrawal was considered and rejected for v1, as it adds significant complexity and user-experience friction to mitigate a narrow window.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	1	1	1
Medium	4	4	4
Low	7	7	4
Informational	6	6	4
Total	18	18	13

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
H-01	Slippage in deposit to strategy allows users to drain the vault	High	Fixed
M-01	Strategy withdrawal mechanism is flawed due to not considering maxRedeem()	Medium	Fixed
M-02	Failed fund processing in CcipAdapter allows arbitrary callers to manipulate deposit amounts via replayFundsReceiving	Medium	Fixed
M-03	Withdrawals can be DoS'ed by frontrunning the nonce usage	Medium	Fixed

M-04	Missing deposit price check allows extracting value from the vault by depositing a depegged asset	Medium	Fixed
L-01	Missing deposit limiter mechanism	Low	Acknowledged
L-02	Malicious strategies cannot be distrusted and they can inflate TVL	Low	Fixed
L-03	<code>_areRemainingSharesRedeemable()</code> calculates best case scenario instead of worst	Low	Acknowledged
L-04	An attacker can force <code>requestWithdrawal(user, 0)</code> to revert if the caller is the only user in the protocol	Low	Fixed
L-05	TVL can temporarily be incorrect	Low	Fixed
L-06	Manager cannot rebalance and limit losses in case of a stablecoin depeg	Low	Acknowledged
L-07	Users can forcefully rebalance funds to default strategy	Low	Fixed
I-01	<code>_setUserRate()</code> could return early instead of reverting	Informational	Acknowledged
I-02	Removing a strategy can be DOSed if not done in a multicall.	Informational	Acknowledged
I-03	Some functions do not need to be payable	Informational	Fixed

I-04	Use constant instead of hardcoded value	Informational	Fixed
I-05	Loops in FundsHandler can be optimized	Informational	Fixed
I-06	Interest accrual can be optimized	Informational	Fixed

High Severity Issues

H-01: Slippage in deposit to strategy allows users to drain the vault

Severity: High	Impact: High	Likelihood: Medium
Files: src/core/Allocator.sol (upon original call from src/core/accounting/StableVault.sol)	Status: Fixed	

Description:

When a user makes a deposit via `StableVault::deposit()`, the funds are deposited to an external strategy to start generating yield, via `Allocator::_depositToStrategy()`.

The deposit to an external strategy is subject to slippage losses. As of the current code, these losses are borne by the vault, and not by the depositor.

This allows any user to completely drain the vault, by repeatedly forcing slippage losses to the vault:

- User deposits 2 wei
- They get 2 Stable Vault shares (assuming vault conversion rate of 1)
- Assuming the strategy's asset/share ratio is > 1 (e.g. 1.2)

- When the allocator deposits 2 wei to the strategy, it will only get 1 share (i.e. it will suffer a slippage loss of ~1 wei of asset)
- Now the Stable Vault has minted 2 shares, but only got 1 asset (due to losing 1 wei upon the deposit)

Recommendations:

Alternative (non exclusive) mitigations:

- Add a minimum deposit amount
- Assign the slippage loss to the user instead of bearing it (i.e. instead of taking the user-passed amount to calculate the amount of shares to mint, use the asset equivalent of the shares received upon the strategy deposit)
- Send funds to a buffer instead of depositing to the strategy upon every Stable Vault deposit (this has the additional bonus of reducing gas cost of deposits)

Customer Response:

What we fixed:

- Introduced a strategy deposit slippage tolerance (single constant across strategies, set to 10 wei in `Allocator.STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE`) to bound the maximum loss any single deposit can incur. Deposits exceeding this slippage revert.
- User's original deposit principal (`originalDepositRay`) is credited from the post-slippage net amount, restoring the invariant principal obligation $\leq$ aggregated assets at deposit time. The pre-fix vault-drain attack (where a user could withdraw principal greater than the asset value backing it) is therefore structurally closed.

Design tradeoff:

- Shares are still minted from the gross deposit amount (Primer Design Decision #10). This preserves the UX invariant that a depositor sees shares matching their input. The delta between gross-share-value and net-principal-credit is classified as the depositor's *interest* claim, not principal — and like all interest claims is withdrawable only against the protocol's real surplus pool (`globalWithdrawableInterest = totalAssets - obligations`), so it cannot dilute other users' principal.

Residual risk:

A depositor's gross-share minting can credit at most 10 wei of phantom interest per deposit, redeemable only from real surplus. With deposit gas cost on Base $\gg$ phantom value, this attack is negative-EV by ~2 orders of magnitude.

Additional mitigation:

For assets where the manager prefers no auto-deposit, setting `defaultStrategyByAsset[asset] = address(0)` routes user deposits to idle balance (`Allocator.sol:194-198`), bypassing strategy slippage entirely until a manager-triggered rebalance.

Fix Review:

Fixed in [PR#154](#).

Limiting the loss to 10 wei makes this attack non economical for stablecoins, as that will be worth at most \$0.000010

This makes the mitigation appropriate for the current intended use of the system

However, that would need to be reviewed if the system is intended to be used with different currencies such as WBTC, where 10 wei would be worth \$0.10 with BTC at \$100,000 – making this attack potentially profitable

In that case, the additional mitigation of keeping the default strategy unset would work.

Medium Severity Issues

M-01: Strategy withdrawal mechanism is flawed due to not considering `maxRedeem()`

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: Allocator.sol	Status: Fixed	

Description:

When a user executes a withdrawal (via `StableVault::executeWithdrawal()`), the Allocator (via `Allocator::withdraw()`) pulls the funds needed for the withdrawal according to the following order:

1. First, from idle balance
2. If #1 was not enough, from funds in the default strategy
3. If #1 + #2 was not enough, from funds in other strategies

```
function withdraw(address asset, uint256 amount) external override
onlyWithdrawer {
    require(amount > 0, Errors.ZeroAmount());
    _validateCanWithdraw(asset);

@1>    uint256 idleBalance = IERC20(asset).balanceOf(address(this));
@2>    if (idleBalance < amount) {
        // Consume from idle balance first
        uint256 amountRemaining = amount - idleBalance;

        // Consume from default strategy
        // Wrap in a try-catch to avoid impact to searching other strategies
@2>        try this.tryWithdrawFromStrategy(asset, amountRemaining,
$storage().defaultStrategyByAsset[asset]) returns (
            uint256 withdrawn
        ) {
            amountRemaining -= withdrawn;
```

```

        } catch {
            emit
StrategyWithdrawalFailed($storage().defaultStrategyByAsset[asset], asset,
amountRemaining);
        }

        // If necessary, pull from remaining strategies
        uint256 length = $storage().assetStrategies[asset].length();
@3>        for (uint256 i = 0; amountRemaining > 0 && i < length; i++) {
            address strategy = $storage().assetStrategies[asset].at(i);
            if (strategy != $storage().defaultStrategyByAsset[asset]) {
@3>                try this.tryWithdrawFromStrategy(asset, amountRemaining,
strategy) returns (uint256 withdrawn) {
                    amountRemaining -= withdrawn;
                } catch {
                    emit StrategyWithdrawalFailed(strategy, asset,
amountRemaining);
                }
            }
        }
        require(amountRemaining == 0, Errors.InsufficientFunds());
    }
    _transferToTransferHelper(asset, amount);
}

```

This is done through the following flow:

[Allocator::tryWithdrawFromStrategy\(\)](#):

```

function tryWithdrawFromStrategy(address asset, uint256 amount, address
strategy)
    external
    onlySelf
    returns (uint256)
{
    uint256 withdrawnAmount;
@4>    uint256 balanceInStrategy =
_getAssetBalanceInStrategy(IERC4626(strategy));
    if (balanceInStrategy > 0) {

```

```
        if (balanceInStrategy > amount) {
            withdrawnAmount = amount;
@5>            _withdrawFromStrategy(asset, withdrawnAmount, address(this),
strategy);
        } else {
@6>            withdrawnAmount = _redeemAllFromStrategy(asset, strategy);
        }
    }
    return withdrawnAmount;
}
```

Allocator::_getAssetBalanceInStrategy()

```
function _getAssetBalanceInStrategy(IERC4626 strategy) internal view returns
(uint256) {
@4.1>    uint256 amount =
strategy.previewRedeem(strategy.balanceOf(address(this)));
    return amount;
}
```

Allocator::_withdrawFromStrategy()

```
function _withdrawFromStrategy(address asset, uint256 amount, address
receiver, address strategy) internal {
    uint256 balanceBefore = IERC20(asset).balanceOf(address(this));
@5.1>    IERC4626(strategy).withdraw({assets: amount, receiver: receiver, owner:
address(this)});
    uint256 balanceAfter = IERC20(asset).balanceOf(address(this));
    require(balanceAfter - balanceBefore >= amount,
Errors.InsufficientAmountOut());
    emit AssetDeallocated(asset, strategy, amount);
}
```

Allocator::_redeemAllFromStrategy()

```
function _redeemAllFromStrategy(address asset, address strategy) internal
returns (uint256) {
@6.1>    uint256 amount = IERC4626(strategy)
        .redeem({
            shares: IERC4626(strategy).balanceOf(address(this)), receiver:
```

```
address(this), owner: address(this)
    });
    emit AssetDeallocated(asset, strategy, amount);
    return amount;
}
```

The main function checks the total value in the strategy via [@4/ @4.1](#), which does so via ERC4626's [previewRedeem\(\)](#).

However, according to the [EIP 4626](#), [previewRedeem\(\)](#):

- MUST NOT account for redemption limits like those returned from `maxRedeem` and should always act as though the redemption would be accepted, regardless if the user has enough shares, etc.
- MUST NOT revert due to vault specific user/global limits. MAY revert due to other conditions that would also cause redeem to revert.

This causes a problem when the shares cannot be withdrawn, as shown in the following example:

- Starting state
 - Idle balance = 0
 - Default strategy balance = 1,000
 - Default strategy withdrawable balance = 800
 - Other strategy [0] balance = 100
- User executes withdrawal for 900
- The function will attempt to redeem the equivalent of 900 assets from the default strategy
- However, since only 800 assets are withdrawable, that attempt will revert
- The function will proceed to attempt to redeem the entirety of the other strategy balance
- The attempt will be successful (as long as all the shares are redeemable)
- But since the withdrawal is for 900 assets and only 100 were obtained, it will revert with the `require(amountRemaining == 0)` check

A similar issue happens on the full withdrawal path (via [@6](#) and [@6.1](#)): the function attempts to redeem all of the available shares, regardless of how many are actually redeemable.

Recommendations:

Cap the withdrawal amount by the strategy's `maxRedeem()`

Customer Response:

The withdrawal path was made `maxWithdraw()`-aware in [PR#149/PR#162](#). The residual zero-`maxWithdraw()` fallback to `previewRedeem`-based logic has been reviewed and accepted: per the ERC-4626 specification, `maxWithdraw()` and `maxRedeem()` share the same semantics (differing only in units), so a strategy returning 0 for `maxWithdraw()` is expected to also return 0 for `maxRedeem()`.

Adding a `maxRedeem()` branch would introduce additional gas cost and code complexity with no practical benefit under this assumption. If a strategy with divergent `maxWithdraw`/`maxRedeem` behavior is onboarded in the future, the Allocator can be upgraded to address it.

Fix Review:

Fixed in [PR#149](#) and [PR#162](#).

M-02: Failed fund processing in CcipAdapter allows arbitrary callers to manipulate deposit amounts via replayFundsReceiving

Severity: Medium	Impact: High	Likelihood: Low
Files:	Status: Fixed	

Description:

When `CcipAdapter.ccipReceive()` processes incoming bridged funds, if `processReceivedFunds()` fails (wrapped in try-catch), the funds remain in the CcipAdapter contract:

```
if (message.destTokenAmounts.length > 0) {
    try this.processReceivedFunds(message.destTokenAmounts) {}
    catch (bytes memory err) {
        for (uint256 i = 0; i < message.destTokenAmounts.length; i++) {
            emit TokenReceptionFailed(...);
        }
        emit BridgedFundsProcessingFailed(...);
    }
}
```

The `replayFundsReceiving()` function in `BaseBridgeAdapter` is intended as a recovery mechanism for these stuck funds, but it has no access control:

```
function replayFundsReceiving(IBridgeAdapter.BridgeAsset[] memory assets)
external virtual override {
    _processReceivedFunds(assets);
}
```

This creates the following issues:

1. Arbitrary amount control: Any caller can invoke `replayFundsReceiving()` with arbitrary amounts (not necessarily matching the original bridged amounts), as long as the adapter holds sufficient balance.
2. Precision loss exploitation: An attacker can call `replayFundsReceiving()` multiple times with tiny amounts (e.g., 2–10 wei per call). When these small amounts are deposited into yield strategies via the Allocator, precision loss during share minting can result in 50–90% of the deposited value being donated to the strategy (depending on the strategy's share-to-asset ratio).
3. TVL manipulation: Until `replayFundsReceiving()` is called, the funds are not reflected in the Allocator's balance, causing a temporary decrease in reported TVL. The balance snapshot sent from the earning chain will not account for these stuck funds.

Recommendations:

We recommend to:

- add access control to `replayFundsReceiving()` to prevent arbitrary manipulation.
- Another less ideal solution is to store the exact amounts that failed to process and enforce that `replayFundsReceiving()` can only replay those specific amounts but this would require writing to storage and always paying the bridge for this additional gas from the source chain in case the call reverts and needs to write to storage.
- If strategy deposit fails, send the funds to the Allocator to remain idle rather than leaving them in the adapter. Here there is a risk the fallback mechanism to fail if the token transfer fails. Unlikely, but if it happens, then the whole transaction would revert and the balance snapshot would not be updated.

Customer Response:

We no longer use the defensive pattern where calls silently revert and sit idle in `CcipAdapter`, instead if the call fails it can be manually retried.

Access control was added to `replayFundsReceiving()` and it will be called in case native or token funds were sent by mistake to the `CcipAdapter`.



Fix Review:

Fixed in [PR#153](#) , [PR#164](#).

M-03: Withdrawals can be DoS'ed by frontrunning the nonce usage

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: WithdrawalPolicy.sol	Status: Fixed	

Description:

Withdrawals are subject to fees. The fees mechanism was designed such that anyone can use it with standard fees, but registered users can get a lower fee through a governance-controlled signer.

The signer signs a message to override the standard fee with a discounted one.

To avoid reuse of the signed message, it includes a nonce, which is marked as used when each withdrawal is processed.

However, a flaw in the `WithdrawalPolicy::applyWithdrawalPolicy()` access control allows anyone to mark a nonce as used, which would make any subsequent call to revert.

Any user can use this to completely DoS discounted withdrawals the following way:

- Monitor the public mempool for `StableVault::executeWithdrawal()` transactions
- Extract the transaction calldata, so that it can assemble a `WithdrawalPolicy::applyWithdrawalPolicy()` with all necessary input
- Front-run the original transaction, marking the nonce as used
- The original transaction will revert on this line:

```
function _verifySignedDiscount(WithdrawalRequest calldata request)
    internal
    view
    returns (address signer, uint256 nonce, uint16 personalFeeBps)
{
    SignedFeeDiscount memory discount = abi.decode(request.data,
(SignedFeeDiscount));
```

```
        require(discount.personalFeeBps <= _getAssetFeeBps(request.assetOut),
Errors.InvalidParameter());
        require(discount.deadline >= block.timestamp, DeadlineExpired());

        signer = _recoverSigner(request, discount);
        require($storage().isSigner[signer], InvalidSignature());
@>    require(!$storage().wasNonceUsed[signer][discount.nonce],
NonceAlreadyUsed());

        return (signer, discount.nonce, discount.personalFeeBps);
    }
```

Recommendations:

Perform access validation on `WithdrawalPolicy::applyWithdrawalPolicy()`.

Customer Response:

`applyWithdrawalPolicy()` is now restricted to `StableVault` and `EarningChainGateway` via the `onlyWithdrawalPolicyApplier` modifier ([PR#134](#)), preventing external callers from consuming signed nonces.

Fix Review:

Fixed in [PR#134](#).

M-04: Missing deposit price check allows extracting value from the vault by depositing a depegged asset

Severity: Medium	Impact: High	Likelihood: Low
Files: StableVault.sol	Status: Fixed	

Description:

Stable Vault is intended to support USD-backed stablecoins. The team intends to list USDC, USDT, and GHO.

The system operates with the underlying assumption that 1 unit of a listed asset equals 1 USD. This allows it to not use an oracle.

However, the downside of this is that the system inherits the security liabilities of all listed assets: if any of the listed assets has a depegging event, the entire value held by the system becomes exposed.

For example:

- Stable Vault has:
 - 1,000 USDT
 - 1,000 USDC
 - 1,000 GHO
- USDC has a depeg event and its market value drops to \$0.01
- A user can
 - Deposit 2,000 USDC (which will cost them \$20) and receive 2,000 shares for it
 - Redeem the 2,000 shares for 1,000 USDT and 1,000 GHO (worth \$2,000)
- This would remove all the valuable assets from the protocol, leaving only the depegged one

As explained by the team, this is intended to be dealt with offchain, by monitoring the market and acting in case this happens.

However, given users can autonomously and permissionlessly deposit and withdraw from the protocol, by the time the monitoring system takes action, the contract's funds might already be depleted.

Recommendations:

Introduce an oracle and check that the asset price is within acceptable bounds when users deposit it.

Additionally, it would increase the security of the system to enforce per-asset limits, such that the exposure to each asset can be controlled.

Customer Response:

The auditor's premise was "no oracle, system assumes 1 listed asset = 1 USD." That premise no longer holds on master.

[PR#157](#) (commit c606d64) introduced the price oracle, which closes the attack at multiple layers:

1. Deposit gate (`StableVault.deposit:222`): `IPriceOracle.validatePrice(asset)` reverts deposits if the oracle's price for the asset falls below `MIN_VALID_PRICE_RAY`. A depegged asset cannot be deposited at face value – the auditor's first attack step fails immediately.
2. Real-value accounting (`LocalBalanceAggregator._getLocalAggregatedBalance`): the protocol's aggregated balance prices each trusted asset through the oracle (`priceRay × balance`). A partially-depegged asset contributes its depegged value, not face value. Solvency checks reflect real value.
3. Withdrawal gate: `StableVault.requestWithdrawal` requires `withdrawalRequestInterestRay ≤ totalAssetsRay – guaranteedObligationsRay`, where `totalAssetsRay` is oracle-priced. A user cannot withdraw more interest than the system actually holds in real value.
4. Distrust as secondary lever: [PR#194](#) (commit f0b65e4) makes `AssetRegistry.distrustAsset` atomic with deposit / swap-output disable. Used for partial depegs that may not breach `MIN_VALID_PRICE_RAY` but still warrant exclusion from accounting.

The auditor's second recommendation (per-asset deposit limits) is not implemented in this finding's scope; it overlaps with L-01 (deposit limiter), which is tracked separately. The oracle-based depeg gate makes per-asset caps redundant for the extraction scenario described, though they remain useful for managing concentration under normal market conditions.

Residual: rapid depeg events that fall below `MIN_VALID_PRICE_RAY` between Chainlink heartbeat updates (worst case heartbeat + grace period). Documented operational mitigation: monitoring alerts trigger distrustAsset (`NO_DELAY`, `OPERATIONAL_ROLE_GUARDIAN`) within minutes - significantly tighter than the auditor's "off-chain monitoring" assumption, since distrust is on-chain and fast.

Fix Review:

Fixed in [PR#157](#). Oracles were introduced.

Low Severity Issues

L-01: Missing deposit limiter mechanism

Severity: Low	Impact: Low	Likelihood: Low
Files: src/core/accounting/StableVault.sol	Status: Acknowledged	

Description:

Stable Vault guarantees the principal deposits of its users, and in the event of any losses, the protocol commits to paying users out of pocket.

With more funds held by Stable Vault, the greater the potential loss in the event of a failed strategy or depeg. Even a small percentage of lost principal could mean millions of dollars.

Recommendations:

We recommend setting a limit on the total amount of deposits Stable Vault can accept. This could be changed by the manager over time and would keep the protocol within a zone where it can fulfill its commitments in the case of an unforeseen event.

Customer Response:

We implement several defensive mechanisms to prevent increased exposure to a particular asset including depeg checks through an oracle and flags in the AssetRegistry.sol contract. We will not implement explicit deposit caps.

L-02: Malicious strategies cannot be distrusted and they can inflate TVL

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/core/Allocator.sol	Status: Fixed	

Description:

If a strategy becomes malicious (gets upgraded to other code, etc) it can drain the whole system because it can inflate TVL by saying allocator shares are worth arbitrary high amount. Inflating TVL would allow users to withdraw their principal and interest or admin to collect remaining surplus while the TVL might not be in connection with reality.

Furthermore, there is no way to remove this malicious strategy because you need `_getAssetBalanceInStrategy()` in the allocator to return 0 in `_removeStrategy()`.

Malicious strategy could block withdrawals, or always return non zero value just to make Allocator removal logic to revert and the only way to stop this is to distrust the asset itself causing the TVL to drop drastically. This action will not allow the admin to remove the strategy but at least it will stop the inflation of the TVL.

Recommendations:

We recommend having a mechanism that will distrust individual strategies and separately try to withdraw any funds from them.

Customer Response:

The TVL-inflation attack vector is closed by [PR#192](#) (commit 6fd4cda), which introduces strategy-level trust:

- `Allocator.distrustStrategy(NO_DELAY, OPERATIONAL guardian)` immediately marks a strategy untrusted, disables deposits to it, and clears the default-strategy slot for its asset.
- `_getTrustedAssetBalances` excludes distrusted strategies from aggregated TVL (`Allocator.sol:544`), so a malicious strategy's reported balance contributes 0 – withdrawals and surplus claims are no longer inflatable through a compromised strategy.

- This cascade mirrors the asset-level distrust cascade introduced in [PR#194](#) for AssetRegistry.

The auditor's secondary concern (a malicious strategy permanently blocking `_removeStrategy` via non-zero balance, plus the awkwardness of `removeStrategy` requiring `isTrusted`) is not closed at the contract level. A spec to decouple removal from distrust was scoped under VA-81 but ultimately canceled – out of scope for v1, with the position that benefits are marginal vs. implementation cost.

The residual cost is operational awkwardness and slot occupancy:

- Distrusted strategies cannot be removed without first being re-trusted (workaround: `trustStrategy` → `deallocate` to zero balance → `removeStrategy`).
- A malicious strategy with stuck balance occupies a strategy slot permanently. The risk is bounded:
 - Distrusted strategies don't contribute to TVL.
 - No new deposits route to them.
 - They can't be promoted to default-strategy.
 - Per-asset slot cap is 15 – generous buffer before exhaustion becomes an operational concern.
 - Adding new strategies is restricted with `HIGH_DELAY` (ADMIN guardian), gating the rate at which slots can be filled by any single actor.

The cost of this gap is consumed strategy slots, not extractable value. Accepted as residual risk for v1.

Fix Review:

Fixed in [PR#192](#), [PR#194](#).

L-03: `_areRemainingSharesRedeemable()` calculates best case scenario instead of worst

Severity: Low	Impact: Low	Likelihood: Low
Files: StableVault.sol	Status: Acknowledged	

Description:

When a user requests a partial withdrawal, `StableVault::requestWithdrawal()` checks whether the user's leftover shares are redeemable for at least 1 wei of asset via `_areRemainingSharesRedeemable()`, and adjusts the withdrawal from partial to full if the leftover shares are worth less than 1 wei of asset.

However, `_areRemainingSharesRedeemable()` checks this through the best case scenario instead of the worst case:

- The team intends to list USDC (6 decimals), USDT (6 dec) and GHO (18 dec)
- The function ensures the share is worth at least 1 wei of an 18 decimals asset
- However, if the protocol runs out of GHO and is only left with USDC and USDT, an amount of shares equivalent to 1 wei of an 18 decimals asset will not be redeemable for an asset with 6 decimals

Recommendations:

Change `Constants.MIN_WITHDRAWABLE_AMOUNT_RAY` from `1e9` (`1e27 / 1e18`) to `1e21` (`1e27 / 1e6`) for the worst case scenario minimum redeemable shares

Customer Response:

Definition of `MIN_WITHDRAWABLE_AMOUNT_RAY = 1e9` is based on the protocol's maximum supported asset precision: 1 wei of an 18-decimal asset in 27-decimal RAY accounting (see `MAX_SUPPORTED_ASSET_DECIMALS=18`). This defines share/accounting dust, not the minimum withdrawable amount for every output asset. With this definition auto-upgrade prevents leaving the dust as defined. Raising it to `1e21` would turn a 6-decimal output limitation into a global



protocol minimum and would unnecessarily block valid high-precision withdrawals. UI should prevent that selection. (AAVE-3566)

L-04: An attacker can force `requestWithdrawal(user, 0)` to revert if the caller is the only user in the protocol

Severity: Low	Impact: Low	Likelihood: Low
Files: /src/core/accounting/Stable Vault.sol#L218	Status: Fixed	

Description:

In case there is only one active position in the entire protocol or there is no other position that accrued interest then an attacker can make a deposit that will reduce current TVL with 1 wei and will cause full withdrawals made shortly after that to revert.

1. User deposits 1000 tokens.
2. Time passes, and 100 interest is accumulated.
3. User calls `requestWithdrawal(user, 0)` to fully withdraw – 1000 principal + 100 interest.
4. Attacker frontruns and makes a deposit (does not matter if 1 wei or 1M) as long as part of it goes to a strategy and rounding down of asset to shares happens there. For example, the attacker deposits 1000 tokens → rounding happens, and the new allocator shares are now worth 999.
5. The user call `requestWithdrawal(user, 0)` attempts to withdraw 1100, but obligations are now 2000, and total assets in the system are 2099, so the interest that will be attempted to be withdrawn is 100, but current available interest is 99, and thus transaction will revert:

Recommendations:

The rounding loss on deposits should be borne by the user and not absorbed by the protocol.

Customer Response:

Closed by [PR#154](#) (commit 2b1f6a0) – the same change that closes Certora H-01. The attack mechanism described required the strategy-deposit rounding loss to be absorbed by protocol obligations (creating an obligations-vs-assets gap that consumed another user's interest pool).

After [PR#154](#), the depositor's `originalDepositRay` is credited from `netDepositAmount` (post-slippage value, `StableVault.sol:248`), so the rounding loss is borne by the depositor's own gross-share-vs-net-principal gap, not by the system's interest pool.

Walking through the auditor's scenario on master: attacker deposits 1000 → strategy returns 999 net → attacker's principal is credited 999. Obligations become 1000 (User A) + 999 (attacker) = 1999; assets become 2099; withdrawable interest = 100 ≥ User A's accumulated 100. No revert.

The strategy slippage tolerance is also bounded at 10 wei per deposit (`STRATEGY_DEPOSIT_SLIPPAGE_TOLERANCE`, `Allocator.sol:62`), so even the maximum-slippage variant of the attack cannot drain more than 10 wei of the attacker's own deposit value, all of which lands as the attacker's "phantom interest," capped against the protocol's actual surplus pool.

Fix Review:

Fixed in [PR#154](#).

L-05: TVL can temporarily be incorrect

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/core/accounting/AccountingChainGateway.sol	Status: Fixed	

Description:

Admin calls `FundsHandler.pushFundsToChain()` to transfer liquidity to the earning chain.

Before sending the cross-chain message, `_updateChainBalanceBeforeBridging()` is executed, increasing `chainBalances.amountRay` by the transferred amount.

This is problematic because, before this liquidity transfer is received on the earning chain, the earning chain could send a balance snapshot to the accounting chain, which could be received first.

Scenario:

1. Strategy #1 on the Accounting chain has 1000, and Strategy #2 on the Earning chain has 1000.
2. Admin transfers 500 liquidity from accounting to earning chain.
3. `chainBalances.amountRay` for earning is 1500, and the Accounting chain has 500.
4. Message is received on the accounting chain that somebody exchanged 1000 IOUs on the earning chain, and now the balance there is 0 (in reality, it should have 500)
5. Then `chainBalances.amountRay` is assigned 0 (happening in `FundsHandler._updateChainBalance()`)
6. The TVL of the protocol is Strategy #1 on the Accounting chain at 500, and Strategy #2 on the Earning chain at 0. Total current TVL is 500, but in reality, it should be 1000.

TVL will return to normal once the earning chain sends another balance snapshot.

Recommendations:

Snapshots transferred between chains introduce lag that can cause discrepancies. Introducing oracles that monitor each chain's Stable Vault TVL would reduce this lag time and contribute to less TVL errors.

Customer Response:

The optimistic-increment race described in the report is fully closed by [PR#145](#) (commit 694d0b2), which removes `_updateChainBalanceBeforeBridging` and its callers. The accounting chain no longer pre-emptively credits in-flight bridge transfers and now relies strictly on oracle-published earning-chain balance snapshots.

[PR#157](#) directly implements the auditor's "introduce oracles that monitor each chain's TVL" recommendation: balance reporting moved from an on-chain push (initiated by the earning chain) to an off-chain Chainlink read of the view function `EarningChainStateProvider.getState()`. The earning chain's on-chain balance-publish entry points were removed (`sendBalanceUpdateWithFeePayer`, `_returnFundsWithBalanceSnapshot`, `_getBalanceSnapshotData`).

The residual property of the system is generic oracle staleness: between Chainlink updates, the accounting chain may temporarily undercount the earning-chain balance. This is bounded by the configured heartbeat (plus the L2 sequencer-uptime grace period for L2 deployments) and surfaced via `ChainBalance.isStale`. Consumers downstream treat stale balance contributions as 0 (`FundsHandler._getAdjustedEarningChainBalanceRay:182-186`), which is the conservative direction - temporary undercounting causes interest-withdrawal reverts (retryable) and surplus-claim reverts (also retryable) rather than over-payment. We treat this bounded residual as acceptable.

Fix Review:

Fixed in [PR#145](#), [PR#157](#).

L-06: Manager cannot rebalance and limit losses in case of a stablecoin depeg

Severity: Low	Impact: Medium	Likelihood: Low
Files: src/periphery/Swapper.sol	Status: Acknowledged	

Description:

Allocator allows the manager to rebalance across different strategies and swap between stablecoins via `Allocator.rebalance()`. This could be done to optimize yield or to manage risk.

The caveat is that the swap function in the Swapper requires swaps to never lose any value that is held by Stable Vault. It does this by requiring slippage to be below a configurable threshold and requiring any slippage to be covered by the manager (the slippage coverage source).

In the event of a major depeg of one of the stablecoins, the manager can choose to wait it out for the stablecoin's price to recover, or to get rid of the token by rebalancing strategies and swapping the depegging stablecoin for something else to prevent future losses.

The issue is that, currently, the only way for the manager to swap a depreciating asset is to pay the slippage, which could be millions or billions of dollars, and might not be realistic. Not swapping the asset on time can lead to additional such losses. The swap can also revert if the slippage percentage exceeds the configured limit.

Recommendations:

Consider a manager function that swaps heavily depegged assets, even with slippage, to limit future losses. It could become callable if the oracle reports that the asset has depegged by a certain percentage or that a certain dollar amount of loss has incurred due to a depeg.

Customer Response:

The 1:1 invariant in `Swapper.executeSwap` is a security property: it prevents a manager-controlled call from extracting arbitrary value through a swap-shaped interface. Oracle-gating or removing this invariant (as the recommendation suggests) would introduce a new attack surface where a manipulated oracle could authorize unfavorable swaps; we consider that strictly worse than the current state.

The current state requires the slippage coverage source to fund the depeg-loss difference, which we acknowledge is impractical for large-magnitude depegs. The durable mitigation is being scoped under VA-96: a dedicated SlippageCoverageVault that pre-positions coverage liquidity rather than relying on a single funded account. This addresses the auditor's concern at the structural level (coverage capacity) without weakening the security invariant.

For v1, we accept the residual risk that swap-based de-risking is impractical for severe depegs. Operational responses available in v1 include disabling user deposits on the affected asset and pausing the relevant flows via guardian roles.

L-07: Users can forcefully rebalance funds to default strategy

Severity: Low	Impact: Low	Likelihood: Medium
Files: Allocator.sol	Status: Fixed	

Description:

When a user executes a withdrawal (via `StableVault::executeWithdrawal()`), the Allocator (via `Allocator::withdraw()`) pulls the funds needed for the withdrawal according to the following order:

1. First, from idle balance
2. If #1 was not enough, from funds in the default strategy
3. If #1 + #2 was not enough, from funds in other strategies

```
function withdraw(address asset, uint256 amount) external override
onlyWithdrawer {
    require(amount > 0, Errors.ZeroAmount());
    _validateCanWithdraw(asset);

@1>    uint256 idleBalance = IERC20(asset).balanceOf(address(this));
@2>    if (idleBalance < amount) {
        // Consume from idle balance first
        uint256 amountRemaining = amount - idleBalance;

        // Consume from default strategy
        // Wrap in a try-catch to avoid impact to searching other strategies
@2>        try this.tryWithdrawFromStrategy(asset, amountRemaining,
$storage().defaultStrategyByAsset[asset]) returns (
            uint256 withdrawn
        ) {
            amountRemaining -= withdrawn;
        } catch {
            emit
```

```
StrategyWithdrawalFailed($storage().defaultStrategyByAsset[asset], asset,
amountRemaining);
    }

    // If necessary, pull from remaining strategies
    uint256 length = $storage().assetStrategies[asset].length();
@3>    for (uint256 i = 0; amountRemaining > 0 && i < length; i++) {
        address strategy = $storage().assetStrategies[asset].at(i);
        if (strategy != $storage().defaultStrategyByAsset[asset]) {
@3>            try this.tryWithdrawFromStrategy(asset, amountRemaining,
strategy) returns (uint256 withdrawn) {
                amountRemaining -= withdrawn;
            } catch {
                emit StrategyWithdrawalFailed(strategy, asset,
amountRemaining);
            }
        }
    }
    require(amountRemaining == 0, Errors.InsufficientFunds());
}
_transferToTransferHelper(asset, amount);
}
```

A malicious user can exploit this design to forcefully rebalance funds from **other strategies** into the **default strategy**.

Consider the scenario below:

- Starting state
 - Idle balance = 0
 - Default strategy = 1,000
 - Other strategies = 1,000
- Attacker requests a withdrawal for 1,000. Before executing the withdrawal, they borrow all available liquidity on the default strategy
- When the Allocator attempts to withdraw from the default strategy, the withdrawal will fail, which will trigger withdrawal from other strategies

- The withdrawal will execute, resulting in the following intermediary state:
 - Idle balance = 0
 - Default strategy = 1,000
 - Other strategies = 0
- Now the user can deposit again, which will send the funds to the default strategy, according to `Allocator::deposit()` logic
- Ending state
 - Idle balance = 0
 - Default strategy = 2,000
 - Other strategies = 0

Recommendations:

This issue is a tradeoff derived from the design decision of:

- Allowing users to freely deposit and withdraw
- Enforcing efficient use of funds
- In a permissionless way

While:

- Maintaining privileged rebalance power

The mitigation to this issue requires a compromise in one of these points.

Customer Response:

Structurally fixed by [PR#279](#) (feat/remove-default-strategy). The attack as described needs (a) a withdrawal path that pulls from the default strategy first, and (b) an auto-deposit path that redirects user redeposits back to that default strategy.

[PR#279](#) removes both legs: the `defaultStrategyByAsset` mapping is dropped (withdrawal iterates `assetStrategies` in insertion order, no preference), and `Allocator.deposit` / `depositAllowIdle` are

replaced with `pullIdle(asset, amount)`, which leaves funds idle – only manager-driven rebalance can move funds into a strategy.

With no targetable strategy concentration on the withdraw path and no auto-redeposit hook on the deposit path, the specific force-rebalance vector this finding describes is gone. The earlier mitigation argument (high `defaultFeeBps` + offchain monitor) is no longer load-bearing for this finding.

Fix Review:

Fixed in [PR#279](#).

Informational Severity Issues

I-01: `_setUserRate()` could return early instead of reverting

Files:

StableVault.sol

Description:

When the privileged caller calls `StableVault::setUserRate()`, the internal function `_setUserRate()` ensures the new rate is different from the one the user had previously, and reverts if it is the same.

However, since `setUserRate()` can take multiple user-newRate pairs at once, if the caller mistakenly includes a user with the same rate, the entire transaction will revert.

Recommendations:

The function could instead return early in this case, which will skip any further action but allow the function call to execute.

Customer Response:

Intentional. Reverting on a redundant rate update surfaces batch configuration mistakes rather than silently skipping them. The error was improved to `RedundantRate(address user, uint256 newPerSecondRate)` for better diagnostics. The manager's offchain system is expected to filter no-op rate changes before submitting the batch.

I-02: Removing a strategy can be DOSed if not done in a multicall.

Files:

src/core/Allocator.sol

Description:

Removing a strategy from the allocator requires the allocator's shares of this strategy to be equal to 0 assets.

If admin rebalances to deallocate all funds from a strategy and in a separate transaction attempt to remove this strategy then a frontrunner can donate 1 wei shares of this vault to make the call revert.

Recommendations:

Use multicall to rebalance and then remove strategies in one transaction.

Customer Response:

The zero-balance requirement on `_removeStrategy` is intentional for correct accounting. The Allocator inherits Multicall and the manager uses OwnedMulticall to atomically batch `deallocate` + `remove` in a single transaction, preventing the 1-wei griefing vector. This is already part of the standard operating model.

I-03: Some functions do not need to be payable

Files:

src/core/accounting/AccountingChainGateway.sol
src/bridging/ccip/CcipAdapter.sol
src/bridging/across/AcrossAdapter.sol

Description:

The payable modifier on a function only matters for receiving native currency sent directly with the function call. Code currently uses the transfer helper pattern where funds are sent there and later in the transaction pulled from another contract to save gas.

The functions that are in between and the one that pulls the funds do not need to be payable because they are not called directly with the native currency. Only the contract function that initially is called needs to be payable and the contract that pulls the funds from the transfer helper needs to have a receive function.

The payable modifier can be removed on the following places:

- AccountingChainGateway.sendPushFundsToChainMessage() - does not touch the native currency
- CcipAdapter.publishMessageToChainWithFeePayer() - calls ITransferHelper(TRANSFER_HELPER).pull() so does not need to be payable.
- AcrossAdapter.publishMessageToChainWithFeePayer() - uses the same pattern as CcipAdapter.

Recommendations:

We recommend removing the payable modifier on the functions mentioned above.

Customer Response:

All three items from the finding are addressed:

1. payable was removed from active bridge adapter interfaces and implementations in [PR#190](#).
2. AcrossAdapter had already been removed entirely in [PR#144](#), making that part moot.

3. AccountingChainGateway.sendPushFundsToChainMessage() had payable removed in [PR#203](#).

Fix Review:

Fixed in [PR#190](#), [PR#144](#), [PR#203](#).

I-04: Use constant instead of hardcoded value

Files:

TransferHelper.sol

Description:

`Constants.sol` defines `NATIVE_CURRENCY = address(0)`.

`TransferHelper::_transfer()`, instead of using the constant, hardcoded `address(0)` for native currency.

The style would be more consistent if it used the constant instead.

Recommendations:

```
function _transfer(address asset, uint256 amount, address destination)
internal {
@>    if (asset == address(0)) {
```

Change `address(0)` for `Constants.NATIVE_CURRENCY`.

Customer Response:

TransferHelper now uses `Constants.NATIVE_CURRENCY` instead of the hardcoded `address(0)`.

Fix Review:

Fixed in [PR#203](#).

I-05: Loops in FundsHandler can be optimized

Files:

src/core/accounting/FundsHandler.sol

Description:

When chain balances are being updated `_updateChainBalance()` loops over all existing chains in storage to make the update.

The loop continues to iterate over remaining chains even after finding the chain it was looking for.

The same pattern can be seen in `_updateChainBalanceBeforeBridging()` and `_decrementChainBalanceSnapshot()`.

Recommendations:

We recommend breaking the loops once the correct chain is found.

Example with `_updateChainBalance()`:

```
if ($storage().chainBalances[i].chainId == chainId) {
    chainExists = true;

    if ($storage().chainBalances[i].nonce < chainBalanceSnapshotNonce) {
        $storage().chainBalances[i].nonce = chainBalanceSnapshotNonce;
        $storage().chainBalances[i].amountRay = snapshotBalanceRay;
    }

    @> break;
}
```

Customer Response:

The snapshot-maintenance loops identified in the finding were removed entirely by the oracle refactor in [PR#157](#), which replaced the old callback-based snapshot model with oracle-based balance reads.

Fix Review:



Fixed in [PR#157](#).

I-06: Interest accrual can be optimized

Files:

StableVault.sol

Description:

Prior to some actions in the Stable Vault (such as `StableVault::deposit()`, `requestWithdrawal()`, etc...), the vault accrues interest to ensure the conversion rate is updated.

This is done via `StableVault::_accrueSubVaultConversionRate()`:

```
function _accrueSubVaultConversionRate(uint256 subVaultId) internal returns
(uint256) {
    uint256 newConversionRate = _previewSubVaultConversionRate(subVaultId);
@>    $storage().subVaultById[subVaultId].conversionRate = newConversionRate;
@>    $storage().subVaultById[subVaultId].lastAccrualTimestamp =
block.timestamp;
    return newConversionRate;
```

If the conversion rate has already been updated in a specific block, the lines marked with an @> above will no-op. However, they will still consume gas from SSTORE in two slots.

Recommendations:

This can be optimized by making `_previewSubVaultConversionRate` return the `secondsSinceLastAccrual`. Then, if the seconds == 0, skip the storage updates.

Customer Response:

Both optimizations are implemented:

1. Same-subvault transfer flow avoids a redundant second accrual by reusing the already-accrued conversion rate ([PR#150](#)).
2. `_accrueSubVaultConversionRate()` now returns early when `secondsSinceLastAccrual == 0`, skipping the redundant storage write ([PR#203](#)).

Fix Review:

Fixed in [PR#150](#) and [PR#203](#).

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.